

计算机与信息工程学院实验报告

姓名	—	学号	
专业		年级	
课程	操作系统	主讲教师	刘扬
实验时间（年月日时）	2025 年 11 月 6 日 10-12 时		
实验地点	计算机大楼 313	辅导教师	刘扬
实验题目	任务调度模拟		
实验目的	在采用多道程序设计的系统中，往往有若干个进程同时处于就绪状态。当就绪状态进程个数大于处理器数时，就必须依照某种策略来决定哪些进程优先占用处理器。本实验模拟在单处理器情况下处理器调度，帮助自己加深了解处理器调度的工作		
实验环境（硬件和软件）	Ubuntu + Python		

一、OS 课程实验

实验内容：

- 1、设计一个按背景材料中指定优先数的优先数调度算法实现处理器调度的程序
- 2、设计一个按时间片轮转法实现处理器调度的程序。

实验步骤：

- 1、按实验指导书内容中对优先数调度算法的描述，编写对应的算法程序，如下所示：

```

# 优先数调度算法模拟
class PCB:
    def __init__(self, name, run_time, priority):
        self.name = name
        self.pointer = None
        self.run_time = run_time
        self.priority = priority
        self.state = "R" # R:就绪, E:结束

    def __str__(self):
        return f"{self.name}\t{self.pointer.name} if self.pointer else '0'\t{self.run_time}\t{self.priority}\t{self.state}"

class PriorityScheduler:
    def __init__(self):
        self.processes = []
        self.head = None

    def initialize_processes(self):
        # 根据提示中的表格初始化进程
        processes_data = [
            ("P1", 2, 1),
            ("P2", 3, 5),
            ("P3", 1, 3),
            ("P4", 2, 4),
            ("P5", 4, 2)
        ]
        for name, run_time, priority in processes_data:
            self.processes.append(PCB(name, run_time, priority))
        # 按优先数从大到小排序并建立队列
        self.processes.sort(key=lambda x: x.priority, reverse=True)
        # 设置指针
        for i in range(len(self.processes) - 1):
            self.processes[i].pointer = self.processes[i + 1]
        # 最后一个进程指针为 0
        self.processes[-1].pointer = None
        # 设置队首
        self.head = self.processes[0]

    def display_queue(self, step=None):
        if step:
            print(f"\n=== 第{step}次调度后进程队列 ===")
        else:
            print("\n=== 初始进程队列 ===")
        print("进程名\t指针\t运行时间\t优先数\t状态")
        print("-----")
        current = self.head

```

```

while current:
    print(current)
    current = current.pointer
def schedule(self):
    step = 1
    print("=== 优先数调度算法模拟 ===")
    self.display_queue()
    while self.head:
        # 选择队首进程运行
        current_process = self.head
        print(f"\n*** 第{step}次调度: 选中进程 {current_process.name} ***")
        # 模拟进程运行: 优先数-1, 运行时间-1
        current_process.priority -= 1
        current_process.run_time -= 1
        print(f"进程 {current_process.name} 运行一次:")
        # 从队列中移除结束的进程
        self.head = current_process.pointer
        current_process.pointer = None
    else:
        # 进程未结束, 需要重新插入队列
        self.head = current_process.pointer
        self.insert_into_queue(current_process)
    # 显示调度后的队列状态
    self.display_queue(step)
    step += 1
    # 检查是否所有进程都结束
    if all(p.state == "E" for p in self.processes):
        print("\n*** 所有进程都已执行完毕! ***")
        break
def insert_into_queue(self, process):
    """按优先数大小将进程插入队列"""
    if not self.head:
        self.head = process
        process.pointer = None
        return
    # 如果新进程优先级最高, 插入队首
    if process.priority > self.head.priority:
        process.pointer = self.head
        self.head = process
        return
    # 寻找插入位置
    current = self.head

```

```

        while current.pointer and current.pointer.priority >= process.priority:
            current = current.pointer
        process.pointer = current.pointer
        current.pointer = process
def main():
    # 创建调度器并初始化
    scheduler = PriorityScheduler()
    scheduler.initialize_processes()
    # 开始调度
    scheduler.schedule()
    print("\n=== 调度过程完成 ===")
if __name__ == "__main__":
    main()

```

2、按实验指导书内容，实现按时间片轮转法实现处理机调度的程序

```

class PCB:
    def __init__(self, name, run_time):
        self.name = name
        self.pointer = None
        self.run_time = run_time # 要求运行时间
        self.ran_time = 0 # 已运行时间，初始为 0
        self.state = "R" # R:就绪, E:结束
    def __str__(self):
        next_process = self.pointer.name if self.pointer else "0"
        return f"{self.name}\t{next_process}\t{self.run_time}\t{self.ran_time}\t{self.state}"

class RoundRobinScheduler:
    def __init__(self):
        self.processes = []
        self.current = None # 标志单元，当前运行的进程
        self.head = None
    def initialize_processes(self):
        # 初始化五个进程，要求运行时间可以任意设定
        processes_data = [
            ("Q1", 2),
            ("Q2", 3),
            ("Q3", 1),
            ("Q4", 4),
            ("Q5", 2)
        ]
        for name, run_time in processes_data:
            self.processes.append(PCB(name, run_time))

```

```

# 建立循环队列
for i in range(len(self.processes)):
    if i == len(self.processes) - 1:
        self.processes[i].pointer = self.processes[0] # 最后一个指向第一个
    else:
        self.processes[i].pointer = self.processes[i + 1]
# 设置队首和当前运行进程
self.head = self.processes[0]
self.current = self.processes[0]
def display_queue(self, step=None, current_process=None):
    if step:
        print(f"\n=== 第{step}次调度后进程队列 ===")
    else:
        print("\n=== 初始进程队列 ===")
    if current_process:
        print(f"当前标志单元指向: {current_process.name}")
    print("进程名\t 下一进程\t 要求运行时间\t 已运行时间\t 状态")
    print("-----")
    # 显示所有进程信息
    for process in self.processes:
        print(process)
def display_circular_queue(self):
    """显示循环队列的连接关系"""
    print("\n 循环队列指针连接关系:")
    if not self.head:
        print("队列为空")
        return
    current = self.head
    visited = set()
    connection = []
    while current and current.name not in visited:
        visited.add(current.name)
        next_name = current.pointer.name if current.pointer else "0"
        connection.append(f"{current.name}→{next_name}")
        current = current.pointer
        if not current or current == self.head: # 回到起点或空指针
            break
    print(" → ".join(connection))
def find_previous_process(self, target_process):
    """找到目标进程的前一个进程"""
    if not self.head or not target_process:
        return None
    current = self.head
    start = self.head

```

```

        self.current = None
        process.pointer = None
        return
# 找到前一个进程
prev_process = self.find_previous_process(process)
if prev_process:
    # 将前一个进程的指针指向被移除进程的下一个进程
    prev_process.pointer = process.pointer
    print(f"更新指针: {prev_process.name}.pointer = {process.pointer.name}")
    # 如果移除的是头节点, 更新头节点
    if process == self.head:
        self.head = process.pointer
        print(f"更新头节点: head = {self.head.name}")
    # 断开被移除进程的指针
    process.pointer = None
    print(f"移除后: {self.head.name}->...->{prev_process.name}->{prev_process.pointer.name}")
if prev_process else 'N/A')")
def get_active_process_count(self):
    """获取就绪状态的进程数量"""
    return sum(1 for p in self.processes if p.state == "R")
def schedule(self):
    step = 1
    print("=== 时间片轮转调度算法模拟 ===")
    self.display_queue()
    self.display_circular_queue()
    while self.get_active_process_count() > 0:
        # 选择当前标志单元指示的进程运行
        current_process = self.current
        print(f"\n*** 第{step}次调度: 选中进程 {current_process.name} ***")
        # 模拟进程运行: 已运行时间+1
        current_process.ran_time += 1
        print(f"进程 {current_process.name} 运行一个时间片:")
        print(f"已运行时间: {current_process.ran_time - 1} → {current_process.ran_time}")
        # 保存下一个进程的指针, 因为 remove_from_queue 可能会改变指针
        next_process = current_process.pointer
        # 判断进程是否结束
        if current_process.ran_time == current_process.run_time:
            current_process.state = "E"
            print(f"进程 {current_process.name} 运行结束!")
            # 从循环队列中移除结束的进程
            self.remove_from_queue(current_process)
            # 如果移除了当前进程, 需要特殊处理 current 指针
            if current_process.state == "E":

```

```

        # 如果队列中还有进程，current 指向下一个进程
        if self.get_active_process_count() > 0:
            self.current = next_process
        else:
            self.current = None
    else:
        print(f"进程 {current_process.name} 尚未结束，等待下一轮调度")
        # 正常情况，current 指向下一个进程
        self.current = current_process.pointer
    # 显示调度后的队列状态
    if self.get_active_process_count() > 0:
        self.display_queue(step, self.current)
        self.display_circular_queue()
        step += 1
    else:
        print("\n*** 所有进程都已执行完毕! ***")
        break

def main():
    # 创建调度器并初始化
    scheduler = RoundRobinScheduler()
    scheduler.initialize_processes()
    # 开始调度
    scheduler.schedule()
    print("\n=== 调度过程完成 ===")
    print("最终进程状态:")
    scheduler.display_queue()
if __name__ == "__main__":
    main()

```

实验数据记录:

1、按优先数调度算法程序的执行结果:

```

(ygkldnherr@WIN-WZY)-[ /mnt/.../study/操作系统 ]
$ python psa.py
=== 优先数调度算法模拟 ===

=== 初始进程队列 ===
进程名  指针    运行时间    优先数  状态
-----
P2      P4      3          5      R
P4      P3      2          4      R
P3      P5      1          3      R
P5      P1      4          2      R
P1      0       2          1      R

*** 第1次调度：选中进程 P2 ***
进程 P2 运行一次：
    优先数：5 → 4
    运行时间：3 → 2

=== 第1次调度后进程队列 ===
进程名  指针    运行时间    优先数  状态
-----
P4      P2      2          4      R
P2      P3      2          4      R
P3      P5      1          3      R
P5      P1      4          2      R
P1      0       2          1      R

*** 第2次调度：选中进程 P4 ***
进程 P4 运行一次：
    优先数：4 → 3
    运行时间：2 → 1

=== 第2次调度后进程队列 ===
进程名  指针    运行时间    优先数  状态
-----
P2      P3      2          4      R
P3      P4      1          3      R
P4      P5      1          3      R
P5      P1      4          2      R
P1      0       2          1      R

*** 第3次调度：选中进程 P2 ***
进程 P2 运行一次：
    优先数：4 → 3
    运行时间：2 → 1

=== 第3次调度后进程队列 ===
进程名  指针    运行时间    优先数  状态
-----
P3      P4      1          3      R
P4      P2      1          3      R
P2      P5      1          3      R
P5      P1      4          2      R
P1      0       2          1      R

*** 第4次调度：选中进程 P3 ***

```

```
*** 第4次调度：选中进程 P3 ***
进程 P3 运行一次：
    优先数：3 → 2
    运行时间：1 → 0
进程 P3 运行结束！

=== 第4次调度后进程队列 ===
进程名  指针    运行时间
-----
P4      P2      1          3
P2      P5      1          3
P5      P1      4          2
P1      0        2          1

*** 第5次调度：选中进程 P4 ***
进程 P4 运行一次：
    优先数：3 → 2
    运行时间：1 → 0
进程 P4 运行结束！

=== 第5次调度后进程队列 ===
进程名  指针    运行时间
-----
P2      P5      1          3
P5      P1      4          2
P1      0        2          1

*** 第6次调度：选中进程 P2 ***
进程 P2 运行一次：
    优先数：3 → 2
    运行时间：1 → 0
进程 P2 运行结束！

=== 第6次调度后进程队列 ===
进程名  指针    运行时间
-----
P5      P1      4          2
P1      0        2          1

*** 第7次调度：选中进程 P5 ***
进程 P5 运行一次：
    优先数：2 → 1
    运行时间：4 → 3

=== 第7次调度后进程队列 ===
进程名  指针    运行时间
-----
P1      P5      2          1
P5      0        3          1

*** 第8次调度：选中进程 P1 ***
进程 P1 运行一次：
    优先数：1 → 0
    运行时间：2 → 1

=== 第8次调度后进程队列 ===
进程名  指针    运行时间
-----
P5      P1      3          1
P1      0        1          0

*** 第9次调度：选中进程 P5 ***
进程 P5 运行一次：
    优先数：1 → 0
    运行时间：3 → 2

=== 第9次调度后进程队列 ===
进程名  指针    运行时间
-----
P1      P5      1          0
P5      0        2          0

*** 第10次调度：选中进程 P1 ***
```

```

***第10次调度：选中进程 P1 ***
进程 P1 运行一次：
    优先数：0 → -1
    运行时间：1 → 0
进程 P1 运行结束！

=== 第10次调度后进程队列 ===
进程名  指针    运行时间  优先数  状态
-----
P5      0      2      0      R

*** 第11次调度：选中进程 P5 ***
进程 P5 运行一次：
    优先数：0 → -1
    运行时间：2 → 1

=== 第11次调度后进程队列 ===
进程名  指针    运行时间  优先数  状态
-----
P5      0      1      -1     R

*** 第12次调度：选中进程 P5 ***
进程 P5 运行一次：
    优先数：-1 → -2
    运行时间：1 → 0
进程 P5 运行结束！

=== 第12次调度后进程队列 ===
进程名  指针    运行时间  优先数  状态
-----

*** 所有进程都已执行完毕！ ***

=== 调度过程完成 ===

(ygkldnherr@WIN-WZY)-[/mnt/.../study/操作系统]
$

```

2、按时间片轮转法实现处理器调度的程序的执行结果：

```

(ygkldnherr@WIN-WZY) [/mnt/.../study/操作系统/实验/实验五]
$ python rrsa.py
=== 时间片轮转调度算法模拟 ===

=== 初始进程队列 ===
进程名  下一进程  要求运行时间  已运行时间  状态
-----
Q1 step Q2 1 2 0 R
Q2: Q3 3 0 R
Q3 print Q4 1 0 R
Q4 break Q5 4 0 R
Q5 Q1 2 0 R

循环队列指针连接关系:
Q1→Q2 → Q2→Q3 → Q3→Q4 → Q4→Q5 → Q5→Q1
s = RoundRobinScheduler()
s.initialize_processes()
*** 第1次调度: 选中进程 Q1 ***
进程 Q1 运行一个时间片:
已运行时间: 0 → 1
进程 Q1 尚未结束, 等待下一轮调度

=== 第1次调度后进程队列 ===
当前标志单元指向: Q2
进程名  下一进程  要求运行时间  已运行时间  状态
-----
Q1 Q2 2 1 R
Q2 Q3 3 0 R
Q3 Q4 1 0 R
Q4 Q5 4 0 R
Q5 Q1 2 0 R
s = RoundRobinScheduler()
s.initialize_processes()

循环队列指针连接关系:
Q1→Q2 → Q2→Q3 → Q3→Q4 → Q4→Q5 → Q5→Q1

*** 第2次调度: 选中进程 Q2 ***
进程 Q2 运行一个时间片:
已运行时间: 0 → 1
进程 Q2 尚未结束, 等待下一轮调度

=== 第2次调度后进程队列 ===
当前标志单元指向: Q3
进程名  下一进程  要求运行时间  已运行时间  状态
-----
Q1 Q2 2 1 R
Q2 Q3 3 1 R
Q3 Q4 1 0 R
Q4 Q5 4 0 R
Q5 Q1 2 0 R
s = RoundRobinScheduler()
s.initialize_processes()

循环队列指针连接关系:
Q1→Q2 → Q2→Q3 → Q3→Q4 → Q4→Q5 → Q5→Q1

*** 第3次调度: 选中进程 Q3 ***
进程 Q3 运行一个时间片:
已运行时间: 0 → 1
进程 Q3 运行结束!

正在移除进程 Q3:
移除前: Q1→...→Q3→Q4
更新指针: Q2.pointer = Q4
移除后: Q1→...→Q2→Q4
s.initialize_processes()

=== 第3次调度后进程队列 ===
当前标志单元指向: Q4
进程名  下一进程  要求运行时间  已运行时间  状态
-----
Q1 Q2 2 1 R
Q2: Q4 3 1 R
Q3 0 1 E
Q4: 调度 Q5 程完成 4 ==> 0 R
Q5 进程: Q1 :> 2 0 R
s.display_queue()

循环队列指针连接关系:
Q1→Q2 → Q2→Q4 → Q4→Q5 → Q5→Q1

*** 第4次调度: 选中进程 Q4 ***
进程 Q4 运行一个时间片:

```

```

*** 第4次调度：选中进程 Q4 ***
进程 Q4 运行一个时间片：
    已运行时间：0 → 1
进程 Q4 尚未结束，等待下一轮调度

== 第4次调度后进程队列 ==
当前标志单元指向：Q5
进程名  下一进程      要求运行时间  已运行时间  状态
-----
Q1      Q2      2      1      R
Q2      Q4      3      1      R
Q3      0       1      1      E
Q4      Q5      4      1      R
Q5      Q1      2      0      R

循环队列指针连接关系：
Q1→Q2 → Q2→Q4 → Q4→Q5 → Q5→Q1

*** 第5次调度：选中进程 Q5 ***
进程 Q5 运行一个时间片：
    已运行时间：0 → 1
进程 Q5 尚未结束，等待下一轮调度

== 第5次调度后进程队列 ==
当前标志单元指向：Q1
进程名  下一进程      要求运行时间  已运行时间  状态
-----
Q1      Q2      2      1      R
Q2      Q4      3      1      R
Q3      0       1      1      E
Q4      Q5      4      1      R
Q5      Q1      2      1      R

循环队列指针连接关系：
Q1→Q2 → Q2→Q4 → Q4→Q5 → Q5→Q1

*** 第6次调度：选中进程 Q1 ***
进程 Q1 运行一个时间片：
    已运行时间：1 → 2
进程 Q1 运行结束！

正在移除进程 Q1：
移除前：Q1→...→Q1→Q2
更新指针：Q5.pointer = Q2
更新头节点：head = Q2
移除后：Q2→...→Q5→Q2

== 第6次调度后进程队列 ==
当前标志单元指向：Q2
进程名  下一进程      要求运行时间  已运行时间  状态
-----
Q1      0       2      2      E
Q2      Q4      3      1      R
Q3      0       1      1      E
Q4      Q5      4      1      R
Q5      Q2      2      1      R

循环队列指针连接关系：
Q2→Q4 → Q4→Q5 → Q5→Q2

*** 第7次调度：选中进程 Q2 ***
进程 Q2 运行一个时间片：
    已运行时间：1 → 2
进程 Q2 尚未结束，等待下一轮调度

== 第7次调度后进程队列 ==
当前标志单元指向：Q4
进程名  下一进程      要求运行时间  已运行时间  状态
-----
Q1      0       2      2      E
Q2      Q4      3      2      R
Q3      0       1      1      E
Q4      Q5      4      1      R
Q5      Q2      2      1      R

循环队列指针连接关系：
Q2→Q4 → Q4→Q5 → Q5→Q2

*** 第8次调度：选中进程 Q4 ***
进程 Q4 运行一个时间片：

```

```
*** 第8次调度：选中进程 Q4 ***
进程 Q4 运行一个时间片：
    已运行时间：1 → 2
进程 Q4 尚未结束，等待下一轮调度

== 第8次调度后进程队列 ==
当前标志单元指向：Q5
进程名  下一进程  要求运行时间  已运行时间  状态
-----
Q1      0          2          2          E
Q2      Q4          3          2          R
Q3      0          1          1          E
Q4      Q5          4          2          R
Q5      Q2          2          1          R

循环队列指针连接关系：
Q2→Q4 → Q4→Q5 → Q5→Q2

*** 第9次调度：选中进程 Q5 ***
进程 Q5 运行一个时间片：
    已运行时间：1 → 2
进程 Q5 运行结束！

正在移除进程 Q5：
移除前：Q2→...→Q5→Q2
更新指针：Q4.pointer = Q2
移除后：Q2→...→Q4→Q2

== 第9次调度后进程队列 ==
当前标志单元指向：Q2
进程名  下一进程  要求运行时间  已运行时间  状态
-----
Q1      0          2          2          E
Q2      Q4          3          2          R
Q3      0          1          1          E
Q4      Q2          4          2          R
Q5      0          2          2          E

循环队列指针连接关系：
Q2→Q4 → Q4→Q2

*** 第10次调度：选中进程 Q2 ***
进程 Q2 运行一个时间片：
    已运行时间：2 → 3
进程 Q2 运行结束！

正在移除进程 Q2：
移除前：Q2→...→Q2→Q4
更新指针：Q4.pointer = Q4
更新头节点：head = Q4
移除后：Q4→...→Q4→Q4

== 第10次调度后进程队列 ==
当前标志单元指向：Q4
进程名  下一进程  要求运行时间  已运行时间  状态
-----
Q1      0          2          2          E
Q2      0          3          3          E
Q3      0          1          1          E
Q4      Q4          4          2          R
Q5      0          2          2          E

循环队列指针连接关系：
Q4→Q4

*** 第11次调度：选中进程 Q4 ***
进程 Q4 运行一个时间片：
    已运行时间：2 → 3
进程 Q4 尚未结束，等待下一轮调度

== 第11次调度后进程队列 ==
当前标志单元指向：Q4
进程名  下一进程  要求运行时间  已运行时间  状态
-----
Q1      0          2          2          E
Q2      0          3          3          E
Q3      0          1          1          E
Q4      Q4          4          3          R
Q5      0          2          2          E

循环队列指针连接关系：
Q4→Q4

*** 第12次调度：选中进程 Q4 ***
进程 Q4 运行一个时间片：
    已运行时间：3 → 4
进程 Q4 运行结束！

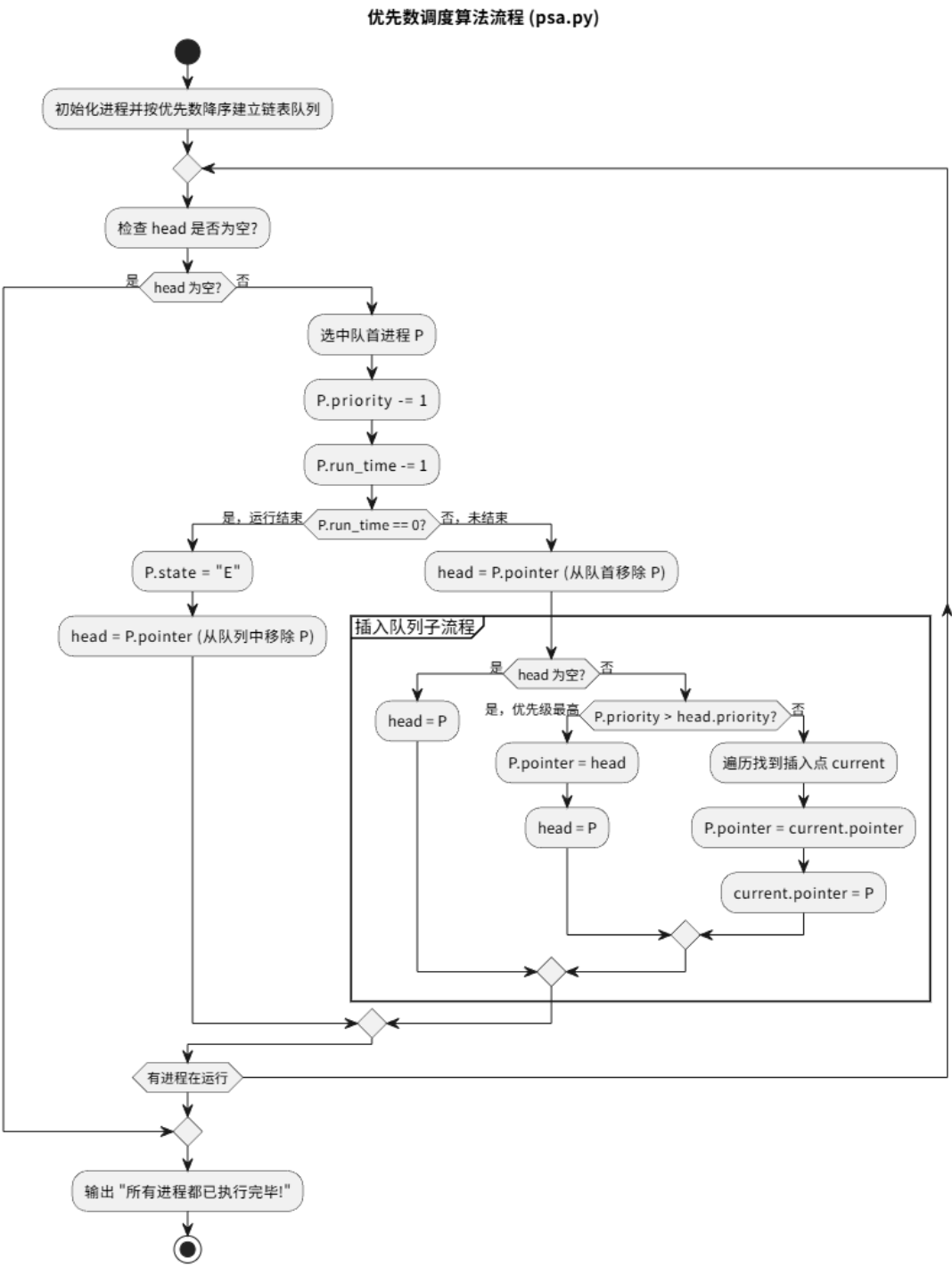
正在移除进程 Q4：
移除前：Q4→...→Q4→Q4
*** 所有进程都已执行完毕！ ***

== 调度过程完成 ==
最终进程状态：

== 初始进程队列 ==
进程名  下一进程  要求运行时间  已运行时间  状态
-----
Q1      0          2          2          E
Q2      0          3          3          E
Q3      0          1          1          E
Q4      0          4          4          E
Q5      0          2          2          E
```

问题讨论：

1、 优先数调度算法的流程图



2、 时间片轮转法实现处理机调度的流程图：

时间片轮转调度算法流程 (rrsa.py)

